COMP2040 – Operating Systems

Lab 04: Synchronization and Deadlocks

Overview

This lab covers Lectures 5 and 6 with one small program. You will watch two threads lose updates to a shared bank account, fix it with a mutex, then watch two transfers that each look correct freeze in a deadlock, and prevent it with a lock order. The story is the same in both weeks: a lock fixes a race, and two locks can stop a program completely.

The starter file `bank.c` has four modes, one per task, and two `TODO` sections. Each account is a struct (`account_t`) with a balance and its own `pthread_mutex_t`; the *C Concepts for Lab 03* guide explains structs and `->`, and Lab 02 Task 5 shows `pthread_create()` and `pthread_join()`. Build the program once, and again after each change:

```
gcc -std=c11 -Wall -Wextra -pthread bank.c -o bank
```

Submission

Upload **one file** to Canvas before the deadline shown there: `StudentID_Lab04.zip`, containing your report `StudentID_Lab04.pdf` and your completed `bank.c`. For example:

```
zip StudentID_Lab04.zip StudentID_Lab04.pdf bank.c
```

In the report, for each task, give the commands, the relevant output (as text) and short answers to the questions.

1 See a race

`./bank race` starts two threads that each deposit 1 into the same account 100 000 times, through `deposit_unsafe()`: read the balance, add 1, write it back. Run it three times.

```
./bank race
```

Include in your report.

(a) The output of your three runs.
(b) Using the read, add and write steps, give an interleaving of the two threads that loses one deposit.
(c) Why does the number of lost deposits change from run to run?

2 Fix it with a mutex

Complete `deposit_safe()` (`TODO 1`) so that the whole read–add–write is one critical section: lock the account's mutex before the read and unlock it after the write. `./bank mutex` must print 200000 on every run.

```
./bank mutex
```

Include in your report.

(a) The output of two runs.
(b) A classmate locks only the write: `tmp = balance + 1; lock; balance = tmp; unlock;`. Why can deposits still be lost?
(c) Why must both threads use the *same* mutex, and why must `main()` join both threads before it prints the balance?

3 Make a deadlock

`./bank deadlock` starts thread A, which moves money from account 1 to 2, and thread B, which moves money from account 2 to 1. Every transfer must hold both accounts' locks, and `transfer_naive()` locks the account it takes money *from* first. `pause_briefly()` waits 1 ms between the two locks so that the problem appears on every run; in real programs it is rare, which makes the bug harder to find. The program freezes, and after 3 seconds without progress a watchdog ends it.

```
./bank deadlock
```

Include in your report.

(a) The output, and the wait-for graph at the moment of the freeze: which thread holds which account's lock, and which lock does each one wait for?
(b) For each of the four necessary conditions, point to the code or the fact in `bank.c` that makes it hold.
(c) The watchdog only knows that no transfer finished for 3 seconds. Why does that not prove a deadlock on its own? What does prove it here?

4 Prevent it with a lock order

Complete `transfer_ordered()` (`TODO 2`) so that every transfer locks the account with the *lower* id first, whichever way the money moves. Then run it 20 times; every run must finish with total money 2000.

```
for i in $(seq 20); do ./bank ordered; done
```

Include in your report.

(a) The output of one run, and how many of the 20 runs finished.
(b) Which of the four conditions does the lock order remove? Explain why the wait-for cycle from Task 3 can no longer form.
(c) In the dining philosophers problem, every philosopher picks up the lower-numbered of their two forks first. Which philosopher now behaves differently from the naive solution, and why does that break the cycle?